Università della Svizzera italiana

Faculty of Informatics

Operating Systems

PintOS: Report Project **#2**

Group #6
Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, March 2, 2024

1 Files changed

- **/pintos/threads/sleepy_thread.h** (*added*)
- **/pintos/devices/timer.c** (*modified*)

2 Changes

/pintos/thread/sleepy_thread.h

- Created this header file declaring **struct sleepy_thread** in order to create a list with **lib/kernel/list.h**. The struct is built to accommodate the requirements of **list.h**, it contains: as a first element the mandatory **struct list_elem elem**, a **struct thread * thread_p** that points to the current thread which needs to be put to sleep, an **int64_t awake_time** (which represents when a thread needs to be woken up in terms of absolute ticks).

/pintos/devices/timer.c

- (*added*) **Includes**:
  - threads/sleepy_thread.h
- (*added*) **Prototypes**:
  - static bool compare_awake_time(const struct list_elem *le1, const struct list_elem *le2, void *aux UNUSED)
- (*added*): **static struct list sleepy_threads_list**; This static global variable is needed to store the threads that are currently blocked and are waiting to be unblocked after **awake_time** ticks.

The variable is in shared memory, so threads calling the method **timer_sleep(int64_t ticks)** can access and modify it.

- (*modified*): **void timer_init(void)**; We initialize the **struct list sleepy_threads_list** here because we wanted it to be done as soon as possible and this is the first method that runs when using timer.c
- (*added*): **static bool compare_awake_time(const struct list_elem *le1, const struct list_elem *le2, void *aux UNUSED)**; This method is needed to use the **list_insert_ordered** method declared in **list.h**. It takes two **struct list_elem *** and checks whether **le1** (list_element 1) needs to be awakened before **le2** (list_element 2). This is done by comparing the two **awake_time** of the elements.
- (*modified*): **void timer_sleep(int64_t ticks)**;
  - Disabled interrupts when entering critical section;
  - For the **current thread**, it computes its **awake_time** by summing the current tick time and the desired sleep duration that is passed as parameter ticks;
  - It inserts in order by awake_time the current thread in the **struct list sleepy_threads_list**
  - It calls the **thread_block()** to block the current thread.
  - When exiting the critical section (thread unblocked) we re-enable interrupts
- (*modified*): **static void timer_interrupt(struct intr_frame args UNUSED)**;
  - Disabled interrupts when entering critical section;
  - Iterating through the **struct list sleepy_threads_list**: we check if there are threads that need to be unblocked by accessing the dedicated field of the list element (awake_time) and if needed we unblock them by calling the method **thread_unblock(struct thread *t)**.
  - When exiting the critical section we re-enable interrupts